

REAL ESTATE PROPERTY SEARCH AND ANALYSIS APPLICATION

Subject: Extreme Computing; Comp 455

Participant List:

Sr. No.	Name	Student Id
1.	Sal Mourad	300126195
2.	Chaahna Chandiramani	300198011
3.	Gautam Garg	300167978
4.	Jack Tse	300146267

Date: 1st December 2024

Introduction

The Real Estate Property Search and Analysis application is a powerful and intuitive platform that empowers users to explore, filter, and analyze real estate listings across the United States. Built using a Flask-based frontend and a Python backend, the application combines advanced search functionalities, robust filtering options, and dynamic sorting capabilities to enhance user experience.

Designed to address common challenges in property exploration, such as inefficient search tools and limited filtering options, this application ensures a seamless and responsive interface for real estate analysis.

This document outlines the system's architecture, dependencies, and detailed data flow, providing insights into the processes and interactions that enable its functionality. A Data Flow Diagram (DFD) is included to visually represent these processes.

System Architecture Overview

- **Static Files:** CSS and JavaScript files are stored in the `/static` directory to style the interface and add interactivity.
- **Templates:** HTML templates used for rendering pages dynamically are housed in the `/templates` directory.
- **Database:** Data is managed using an SQLite database, named `real_estate_new.db`, ensuring efficient query execution and scalability.

These architectural components ensure the application remains lightweight, modular, and scalable for future enhancements.

Dependencies

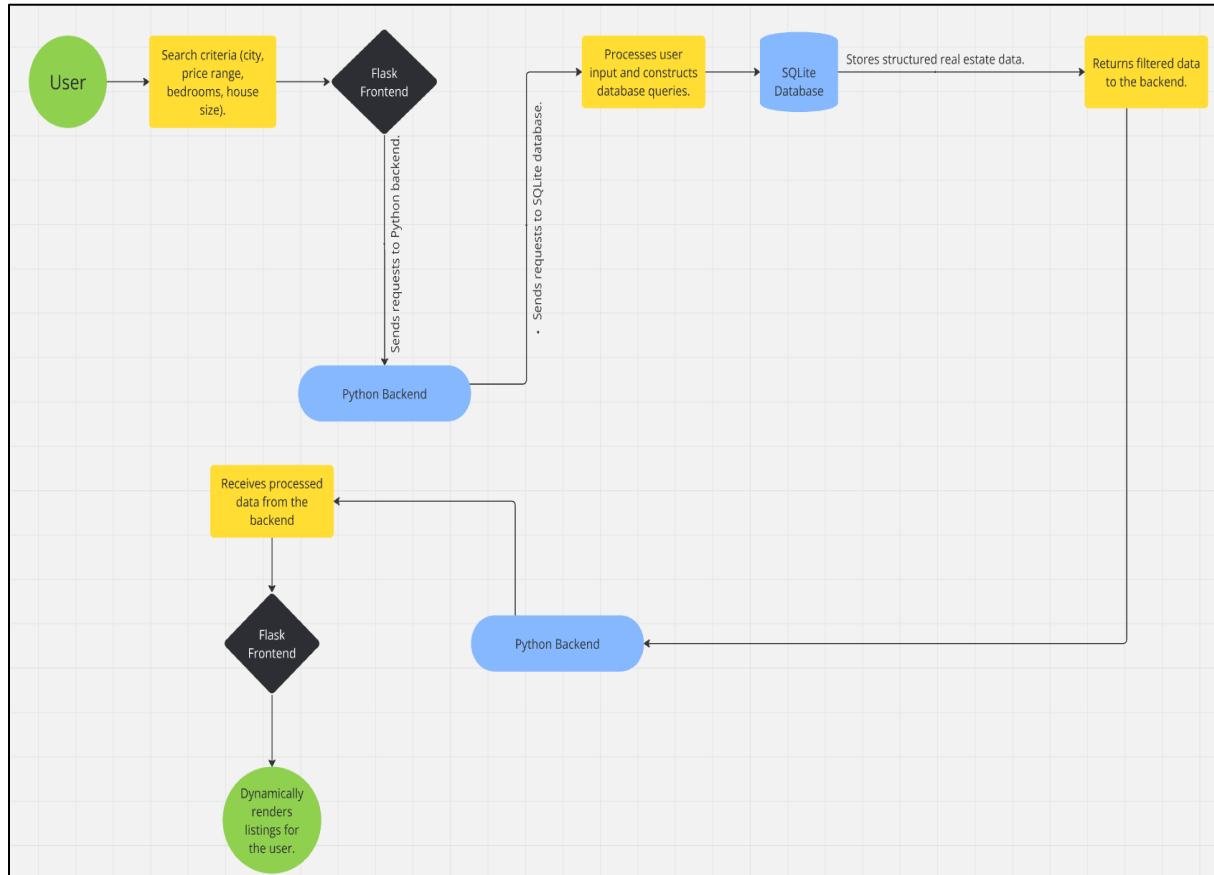
- **Backend:** Python libraries including:
 - **Flask:** For request handling and routing.
 - **SQLAlchemy:** For object-relational mapping (ORM) and database interaction.
 - **pandas:** For efficient data manipulation and analysis.
- **Frontend:** Built using basic HTML, CSS, and JavaScript to create a responsive and interactive user interface.

These dependencies enable the application to process large datasets efficiently while delivering an intuitive and visually appealing experience.

Design Section

Data Flow Diagram (DFD)

The following diagram illustrates the data flow within the Real Estate Property Search application. It highlights the interaction between user inputs, backend processes, database operations, and frontend rendering.



Detailed Workflow Description

1. User Interaction

- **Search Functionality:** Users can search for properties by city, state, price range, number of bedrooms, and house size in square feet.
- **Filtering Options:** Results can be refined using filters such as price range, number of bedrooms/bathrooms, and property size.
- **Sorting Options:** Listings can be sorted by price (low-to-high or high-to-low).

2. Data Processing

- Search queries and filter selections are processed by the Flask frontend, which communicates with the Python backend.
- The backend handles business logic, constructs SQL queries, and fetches data based on user inputs.

3. Database Operations

- Data is retrieved from the SQLite database (real_estate_new.db), sourced from a structured dataset provided by Kaggle.
- The database ensures efficient retrieval, even for large datasets, with optimized indexing for performance.

4. Results Display

- Processed data is returned to the frontend, formatted, and displayed dynamically using HTML templates.
- Interactive features, such as autocomplete and visual trends, enhance user engagement and analysis capabilities.

Conclusion

Real estate is a key driver of the urban economy.[1] It is important to analyze Real Estate data.

The Real Estate Property Search and Analysis application is a robust solution for exploring and analyzing real estate listings. Its streamlined architecture, intuitive interface, and efficient data flow processes make it a valuable tool for users seeking accurate and comprehensive property information.

The inclusion of a Data Flow Diagram emphasizes the seamless integration of user inputs, backend processing, and database interactions, ensuring an optimal user experience.

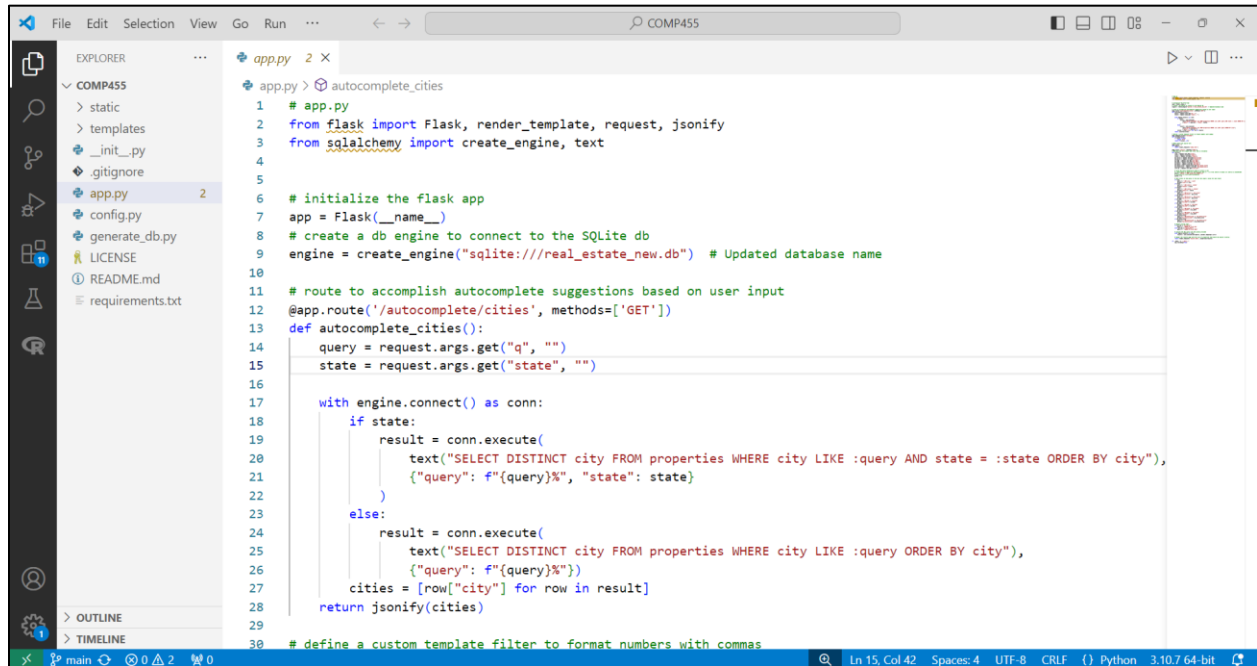
Future enhancements could include API integration for real-time updates, support for international listings, and advanced visualization tools, further solidifying its utility in the real estate sector.

References

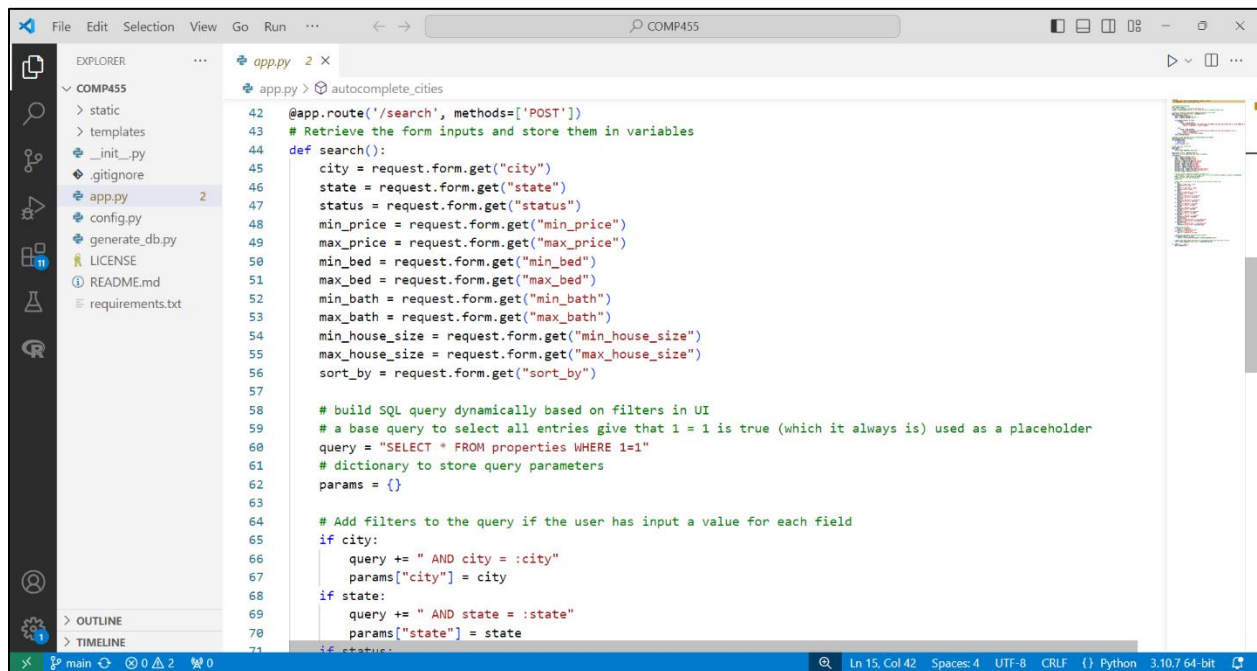
- 1] Ahramovich, A. (2023, August). *Machine Learning in Real Estate: an Innovation Catalyst*.
Www.itransition.com. <https://www.itransition.com/machine-learning/real-estate>
- 2] Cameron. (2024, October 24). *How AI and Data Analytics Are Transforming Property Valuation Models in Real Estate - Estate Agent Today*. Estate Agent Today.
<https://www.estateagenttoday.co.uk/article/2024/10/how-ai-and-data-analytics-are-transforming-property-valuation-models-in-real-estate/>
- 3] Gibb, K., & Hoesli, M. (2003). Developments in Urban Housing and Property Markets. *Urban Studies*, 40(5-6), 887–896. <https://doi.org/10.1080/0042098032000074227>
- 4] Huang, H., Gartner, G., Krisp, J. M., Raubal, M., & Van de Weghe, N. (2018). Location based services: ongoing evolution and research agenda. *Journal of Location Based Services*, 12(2), 63–93. <https://doi.org/10.1080/17489725.2018.1508763>
- 5] Siniak, N., Kauko, T., Shavrov, S., & Marina, N. (2020). The impact of proptech on real estate industry growth. *IOP Conference Series: Materials Science and Engineering*, 869(6), 062041. <https://doi.org/10.1088/1757-899x/869/6/062041>

Appendix

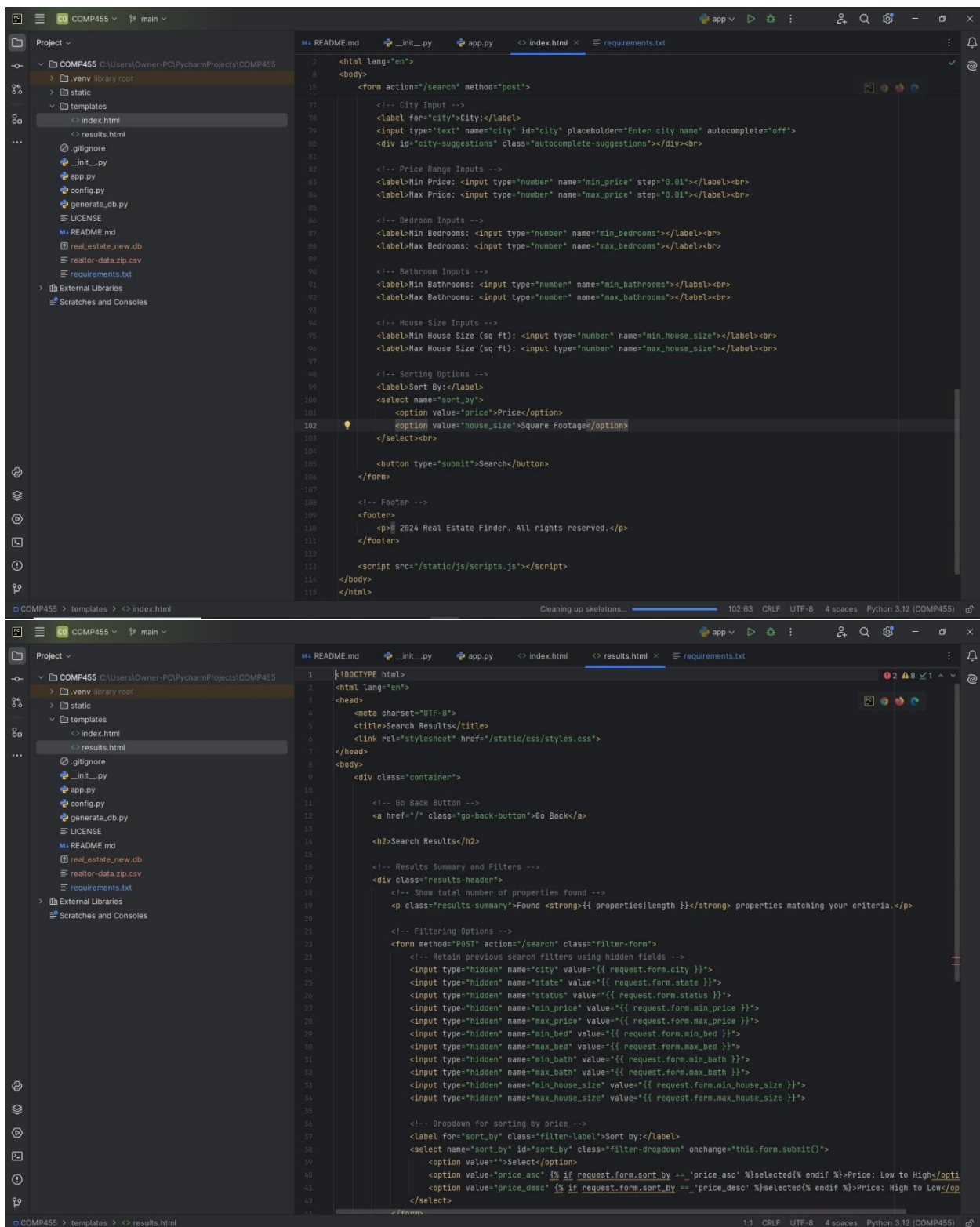
Code Snippets



```
1 # app.py
2 from flask import Flask, render_template, request, jsonify
3 from sqlalchemy import create_engine, text
4
5
6 # initialize the flask app
7 app = Flask(__name__)
8 # create a db engine to connect to the SQLite db
9 engine = create_engine("sqlite:///real_estate_new.db") # Updated database name
10
11 # route to accomplish autocomplete suggestions based on user input
12 @app.route('/autocomplete/cities', methods=['GET'])
13 def autocomplete_cities():
14     query = request.args.get("q", "")
15     state = request.args.get("state", "")
16
17     with engine.connect() as conn:
18         if state:
19             result = conn.execute(
20                 text("SELECT DISTINCT city FROM properties WHERE city LIKE :query AND state = :state ORDER BY city"),
21                 {"query": f"{query}%", "state": state}
22             )
23         else:
24             result = conn.execute(
25                 text("SELECT DISTINCT city FROM properties WHERE city LIKE :query ORDER BY city"),
26                 {"query": f"{query}%"}
27             )
28     cities = [row["city"] for row in result]
29     return jsonify(cities)
30
31 # define a custom template filter to format numbers with commas
```



```
42 @app.route('/search', methods=['POST'])
43 # Retrieve the form inputs and store them in variables
44 def search():
45     city = request.form.get("city")
46     state = request.form.get("state")
47     status = request.form.get("status")
48     min_price = request.form.get("min_price")
49     max_price = request.form.get("max_price")
50     min_bed = request.form.get("min_bed")
51     max_bed = request.form.get("max_bed")
52     min_bath = request.form.get("min_bath")
53     max_bath = request.form.get("max_bath")
54     min_house_size = request.form.get("min_house_size")
55     max_house_size = request.form.get("max_house_size")
56     sort_by = request.form.get("sort_by")
57
58     # build SQL query dynamically based on filters in UI
59     # a base query to select all entries give that 1 = 1 is true (which it always is) used as a placeholder
60     query = "SELECT * FROM properties WHERE 1=1"
61     # dictionary to store query parameters
62     params = {}
63
64     # Add filters to the query if the user has input a value for each field
65     if city:
66         query += " AND city = :city"
67         params["city"] = city
68     if state:
69         query += " AND state = :state"
70         params["state"] = state
71     if status:
```



Input/Output

Search Real Estate Properties In The United States

Search through thousands of properties in the United States to find your perfect match.

State:

California

City:

san

- San Francisco
- San Gabriel
- San Gregorio
- San Jacinto

Feature 1: the program can provide auto correction for city names it retrieved from the dataset

Max Bathrooms:

1

Min House Size (sq ft):

Max House Size (sq ft):

Sort By:

Price

Search

Feature 2: the program can provide results even though some fields are left blank.

Sort
by:

Price: Low to High

Select

Price: Low to High

Price: High to Low

Feature 3: the program can sort from high to low or the other way.

Example Input 1:

State: California

Max Bathroom: 1

Sort By: Price Low to High

Example Output 1

Go Back

Search Results

Found 714 properties matching your criteria.

Sort by: Price: Low to High

City: San Francisco

Street: 223067

State: California

Zip Code: 94122

Price: \$213,389.00

Bedrooms: 2.0

Bathrooms: 1.0

House Size: 854.0 sq ft

City: San Francisco

Street: 1025277

State: California

Zip Code: 94158

Price: \$303,285.00

Bedrooms: 1.0

Bathrooms: 1.0

House Size: 730.0 sq ft

City: San Francisco

Street: 478241

State: California

Zip Code: 94109

Price: \$314,156.00

Bedrooms: 1.0

Bathrooms: 1.0

House Size: 500.0 sq ft

City: San Francisco

Street: 1711663

State: California

Zip Code: 94109

Price: \$353,803.00

Bedrooms: 1.0

Bathrooms: 1.0

House Size: 705.0 sq ft

City: San Francisco

Street: 1286838

State: California

Zip Code: 94124

Price: \$359,000.00

Bedrooms: 1.0

Bathrooms: 1.0

House Size: 621.0 sq ft

City: San Francisco

Street: 1286839

State: California

Zip Code: 94124

Price: \$380,000.00

Bedrooms: 1.0

Bathrooms: 1.0

House Size: 561.0 sq ft

City: San Francisco

Street: 1395621

State: California

Zip Code: 94131

Price: \$391,120.00

Bedrooms: 1.0

Bathrooms: 1.0

House Size: 684.0 sq ft

City: San Francisco

Street: 334806

State: California

Zip Code: 94103

Price: \$392,616.00

Bedrooms: 2.0

Bathrooms: 2.0

House Size: 913.0 sq ft

City: San Francisco

Street: 1750786

State: California

State: California

City: San Francisco

Street: 223066

State: California

State: California

City: San Francisco

Street: 1725396

State: California

State: California

City: San Francisco

Street: 1248962

State: California

State: California

Example Input 2:

State: Georgia

City: Jefferson

Min Price: 100

Max Price: 5000000

Min Bedrooms: 1

Max Bedrooms: 5

Min Bathrooms: 1

Max Bathrooms: 5

Min House Size (sq ft): 1

Max House Size (sq ft): 5000

Sort By: Square Footage High to Low

Example Output 2:

[Go Back](#)

Search Results

Found **48** properties matching your criteria.

Sort by: Price: High to Low

City: Jefferson Street: 904198 State: Georgia Zip Code: 30549 Price: \$1,500,000.00 Bedrooms: 4.0 Bathrooms: 4.0 House Size: 4267.0 sq ft	City: Jefferson Street: 328030 State: Georgia Zip Code: 30549 Price: \$829,000.00 Bedrooms: 4.0 Bathrooms: 4.0 House Size: 3700.0 sq ft	City: Jefferson Street: 1639712 State: Georgia Zip Code: 30549 Price: \$749,900.00 Bedrooms: 5.0 Bathrooms: 4.0 House Size: 3613.0 sq ft	City: Jefferson Street: 194485 State: Georgia Zip Code: 30549 Price: \$725,000.00 Bedrooms: 4.0 Bathrooms: 4.0 House Size: 2317.0 sq ft
City: Jefferson Street: 1233043 State: Georgia Zip Code: 30549 Price: \$719,950.00 Bedrooms: 4.0 Bathrooms: 5.0 House Size: 3964.0 sq ft	City: Jefferson Street: 533273 State: Georgia Zip Code: 30549 Price: \$695,000.00 Bedrooms: 4.0 Bathrooms: 6.0 House Size: 3512.0 sq ft	City: Jefferson Street: 1437655 State: Georgia Zip Code: 30549 Price: \$589,900.00 Bedrooms: 6.0 Bathrooms: 4.0 House Size: 4731.0 sq ft	City: Jefferson Street: 1451327 State: Georgia Zip Code: 30549 Price: \$585,000.00 Bedrooms: 6.0 Bathrooms: 4.0 House Size: 3604.0 sq ft
City: Jefferson Street: 1233178 State: Georgia	City: Jefferson Street: 1222573 State: Georgia	City: Jefferson Street: 1819198 State: Georgia	City: Jefferson Street: 1810699 State: Georgia